

A High-speed Unsigned 32-bit Multiplier Based on Booth-encoder and Wallace-tree Modifications

Xuan-Vy Luu, Trong-Thuc Hoang, and Trong-Tu Bui

Faculty of Electronics and Telecommunications (FETEL)

The University of Science

Ho Chi Minh City, Vietnam

{lxvy, htthuc, btuu}@fetel.hcmus.edu.vn

Anh-Vu Dinh-Duc

University of Information Technology (VNU-HCM)

Ho Chi Minh City, Vietnam

anhvu@uit.edu.vn

Abstract—The delay of the multiplier plays a critical role in many high-speed implementations and processors such as RISC, DSP, and image processing cores, etc. In this paper, a design of unsigned 32-bit multiplier is proposed, aiming to achieve the best timing performance with an appropriate area. The proposed architecture consists of a modified Radix-4 Booth encoder, a modified Wallace Tree adder, and a Carry Look Ahead adder. The design has been verified successfully on DE2-115 and then synthesized to ASIC implementation. The FPGA-based experimental result shows that it has the resources of 1788 ALUTs. The synthesized result occupies an area of 58.28 mm^2 with 4.13 ns total delay (i.e. 242.13MHz maximum frequency).

I. INTRODUCTION

Multiplier circuit is an indispensable part of processor implementations. Many kinds of multiplier designs have been proposed in order to overcome various designers' requirements such as small area, high speed, or low energy consumption, etc. Optimizing multiplier has attracted strong attention from researchers over the time. There are many implementation techniques that have been proposed such as Booth encoder [1], Wallace tree adder [2], Dadda tree adder [3], Carry-save adder [4], array multiplier [5], Modified Booth Encoder [6], etc. Generally, a multiplier implementation can be divided into three steps as can be seen in Fig. 1. In the first step, partial product generation transforms the multiplier to a number of products. In the second step of partial product addition, the process can be described as summing up all the partial products until the last two rows. Various techniques have been presented in order to reduce the amount of additions along with speeding up the processing by parallel designs. When all partial products are reduced to sum and carry words, the final step adds the last two rows to generate the final multiplication result.

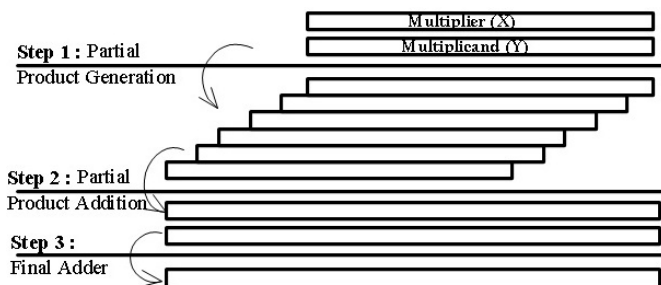


Fig. 1: Three steps of a typical parallel multiplier.

In the first step of multiplier design, 2-input AND gates can be used [1] due to its advantage of time-consuming. However, the drawback is that the number of partial products is high, which leads to a significant delay in the second step. Thus, the main objective in first step is to reduce the amount of partial products; the cost of time is the secondary objective in this step. The Booth encoder [1], [7] is one of the most common methods which have been developed for that purpose. Based on [8], parallel multipliers which deploy Booth Encoder give an advantage of processing time but in the cost of area. In Booth-based designs, various radices can be applied such as radix-4, radix-8, radix-16. Radix-32 or higher are not common due to its high complexity. Both radix-16 and radix-8 have a 2's complement converter and an addition circuit, which cause disadvantages in area and delay compared to radix-4 architectures. However, although radix-4 Booth encoder does not need an addition circuit, it only reduces the number of partial products 2 times, while the radix-8 and radix-16 can reduce those 3 and 4 times respectively. In the comparison between radix-16 and radix-8, the fact that radix-16 is preferred because it can reduce the number of partial products more than that of radix-8 with an equivalent area cost. Although radix-16 approach has strong advantages over the radix-8, it is hard to have a clear decision of which is better between radix-16 and radix-4 designs. D. Villeger and V. G. Oklobdzija [7] have shown that the radix-4 Booth Multiplier has a poor delay performance. However, the performance depends on the schematic as given by L. H. Hiung and K. L. S. Swee [8]. Therefore, radix-4 and radix-16 are the two best Booth-based designs which aim to optimize the speed. In this research, the authors present the proposed design applied both radices to give a clear comparison.

In the second step, a tree adder is deployed to sum all the partial products. There are two common approaches with various advancements, they are Wallace Tree Adder (WTA) approach [2] and Dadda Tree Adder (DTA) approach [3]. The WTA-based methods are developed based on optimizing the compressor units such as 4-2 compressors [4] and/or 5-2 compressors [9], while the DTA-based approaches use 3-2 compressors as its basic elements. A strong advantage of WTA algorithm is that it can generate the final two rows with only few row additions. However, the complexity of each row-addition is high, which leads to a high cost of timing and resources for each row-addition process. On the other hand, although each row-addition in DTA algorithm is simpler, DTA-based designs have more row additions than those of the WTA-based architectures.

In the final step, two last rows are added together to compute the final multiplier result. Many different kinds of high-speed adder have been presented such as Carry-save Adder [4], Carry Look Ahead Adder (CLA) [10], [11], etc. However, it can be proven that there is a part of the final multiplier result can be computed at the end of each row-addition processing in step 2. Therefore, the typical final step can be done during the processing of its previous step, thereby giving rises to an improvement in processing time.

In this paper, we present a high-speed unsigned 32-bit multiplier based on the modifications of radix-4 Booth encoder and WTA, which benefits from sharing processing and resources. The proposed schematic is proven that it is suited best for radix-4 Booth algorithm rather than radix-16 approach. The modified radix-4 Booth encoder reduces the processing time significantly in the comparison with traditional designs. The modified WTA is a combination of WTA and DTA. It has an advantage that the 30-bit LSB of the multiplication result can be computed earlier in step 2. As consequences, step 3 needs only a 34-bit CLA to compute the final result. Furthermore, the design is optimized in schematic in order to achieve the best performances. The experimental result shows that the total processing time of the proposed multiplier approximates to the delay of a 64-bit CLA. In order to verify the function and timing of the proposed multiplier, the implementation is tested on DE2-115 Altera development kit with Cyclone IV FPGA, and synthesized with 90nm CMOS technology.

The remainder of this paper are as follows. Section II presents the proposed fundamental algorithms including modified Radix-4 Booth encoder, Carry Look Ahead adder, and modified Wallace tree adder. Section III shows the experimental results and the comparison of the proposed design to the others. Conclusions are given in Section IV.

II. FUNDAMENTAL ALGORITHMS

A. Proposed Radix-4 Modified Booth Encoder

The radix-4 design should be chosen rather than radix-16 design because of the following reasons. Radix-4 requires only a 2's complement converter in order to encode, while radix-16 needs another addition circuit. As a result, radix-4 design has smaller area, better delay, and simpler implementation than those of the radix-16 design. However, the number of partial products created by the radix-4 design is twice than those created by the radix-16 design. It can be proven that with our modified Booth encoder (MBE) method, radix-4 design gives only a slightly increase in resources, but its timing performance is 3 times better than that of the radix-16 approach. The idea of the MBE is that moves the complement carry into the Wallace tree adder in the following step. As a result, the processing time of MBE is reduced significantly. "A fast method to find two's complements" has been presented by J. Y. Kang [12] to speed up the processing time of 2's complement converter in Booth algorithm. However, the advantage of the Kang's design cannot overcome the idea which moves complement carry into the next step as in the proposed MBE.

TABLE I: The operations of the proposed modified Radix-4 Booth Encoder.

Input Data Multiplier bits [3 bits]	Operation	Output Data	
		Generated Partial Products [33 bits]	Complement Carry [2 bits]
000	$Y * 0$	0	0 0
001	$Y * 1$	Y	0 0
010	$Y * 1$	Y	0 0
011	$Y * 2$	$Y \ll 1$	0 0
100	$Y * (-2)$	$(\sim Y) \ll 1$	1 0
101	$Y * (-1)$	$\sim Y$	0 1
110	$Y * (-1)$	$\sim Y$	0 1
111	$Y * 0$	0	0 0

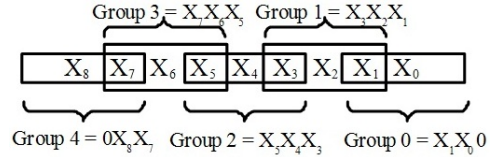


Fig. 2: Bit-grouping of the radix-4 Booth Encoder applied for 8-bit data.

The proposed modified Radix-4 Booth encoder (MBE) is deployed in the first step of partial product generation. The general idea of radix-4 Booth encoder is that it only takes every second column of the multiplier term and multiplies with ± 1 , ± 2 , or 0, instead of shifting and adding for every column and multiplying by 1 or 0 in the traditional way. In each multiplier term, three bits are grouped as a block, and each block overlaps the previous one by one bit as can be seen in the example in Fig. 2. For the a term of 3-bit group, there are 8 possible cases to generate a partial product as shown in Table I.

By using radix-4 Booth encoder, the number of partial products is reduced by half compared to the traditional way. However, due to the unsigned multiplier, one addition signed bit must be added in order to compute the operations. Hence, there are $N/2 + 1$ products with N -bit multiplier.

According to Table I, it can be seen that the worst case delay in radix-4 Booth encoder given by the case of "100" 3-bit group when Y multiply by -2 . Traditionally, the output data needs a 2's complement converter and 1-bit shifting circuit to compute the calculation. But the 2's complement converter consists of NOT logic and one addition circuit, which leads to the delay increasing by the addition circuit. The proposed MBE can overcome the issue by moving the addition part to the Wallace Tree Adder in the following step. Then later, the addition is composed with other additions in the next step to make a best optimization. For example, Z is called the partial product in the worst case delay. Therefore, Z can be computed:

$$\begin{aligned}
 Z &= Y * (-2) \\
 &= (-Y) * 2 \\
 &= (\sim Y + 1) \ll 1 \\
 &= (\sim Y \ll 1) + 2
 \end{aligned} \tag{1}$$

The proposed MBE computes only $(\sim Y \ll 1)$, and the $+2$ is moved to the next step. As a result, the output data consist of two parts: a 33-bit partial product and a 2-bit complement carry as can be seen in Table I. The complement carry is computed later during the products summing process in the second step. To conclude, the processing time for radix-4 MBE in step 1 is optimized.

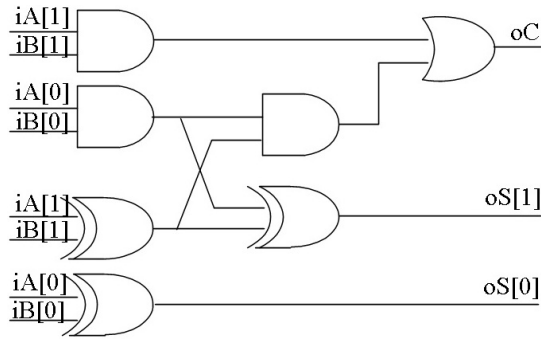


Fig. 3: The schematic of 2-bit half CLA.

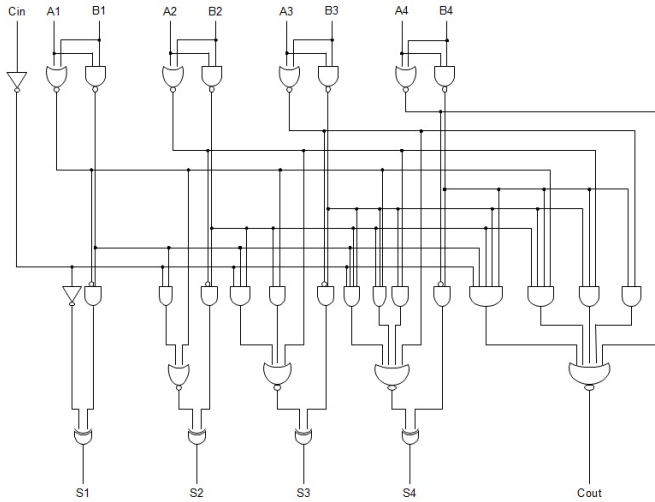


Fig. 4: The schematic of 4-bit full CLA.

B. Carry Look Ahead Adder

Carry Look Ahead adder (CLA) is used in both the WTA in step 2 and the final addition in step 3. There are two kinds of CLA that used in the paper: CLA without carry in called half CLA, and CLA with carry in called full CLA. The examples schematics of 2-bit half and 4-bit full CLA are shown in Fig. 3 and Fig. 4, respectively. The proposed modified WTA in step 2 uses half CLAs along with 3-2 and 2-2 compressors. The step 3, final addition, uses a 34-bit CLA which is built by 2-bit half and 4-bit full CLAs as can be seen in Fig. 5.

C. Modified Wallace Tree Adder

Wallace tree adder is a technique that can sum all partial products with an advantage of processing time. From the MBE in the previous step, there are partial products with their complement carry that need to be summed. Then, the process of proposed modified Wallace tree adder (MWTA) can be divided into 3 computation areas as can be seen in Fig. 6. They are signed area, MSB area, and LSB area. The signed area is the extended signed-bit of the partial products. The LSB computation area composes of LSB-bit of partial products with complement carry bits from the previous step. It can compute directly the LSB-bit of the final multiplier result during the MWTA process. Unlike the LSB computation area,

The MSB area cannot calculate directly the MSB-bit of the final multiplier result; it needs a final adder to perform the computation. As can be seen in Fig. 6, the LSB and MSB computation areas are 6-bit and 10-bit wide, respectively. For the case of 32-bit multiplier, they are 30-bit and 24-bit wide for LSB and MSB computation areas, respectively.

1) *Signed Area Computation*: The important property of the signed computation area is that it can be an extended of "0" or "1" only. Therefore, there are 4 cases of summing two components in the signed area as shown in Fig. 7. Let X and Y be two components in signed area, and Y is shifted by n -bit to X, and Z is the summing result of X and Y. Then, the 4 cases of Z can be seen in Fig. 7. As a result, the formula of Z is as follows.

$$\begin{aligned} Z_{n-1} &= Z_{n-2} = \dots = Z_0 = X_n \\ Z_n &= X_n(xor)Y_n \\ Z_{n+1} &= Z_{n+2} = \dots = Z_m = X_n(or)Y_n \end{aligned} \quad (2)$$

where, Y is shifted by n -bit to X and m is the maximum bits of X and Y as shown in Fig. 7. The n number is called the mark of X to Y.

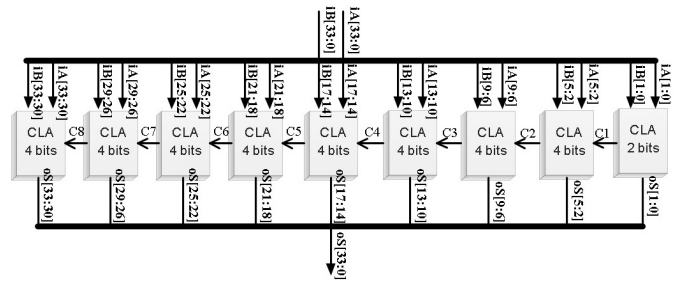


Fig. 5: The 34-bit CLA based on the basic elements of 2-bit half and 4-bit full CLAs.

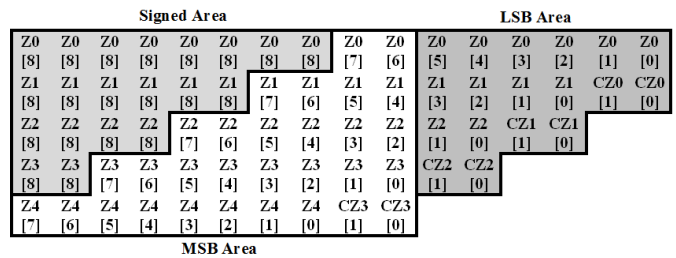


Fig. 6: Example of MWTA for 8-bit partial products.

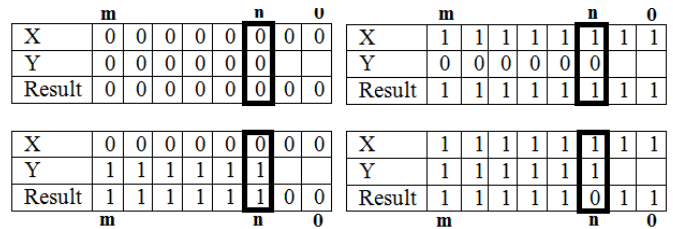


Fig. 7: Example of summing two components in signed area.

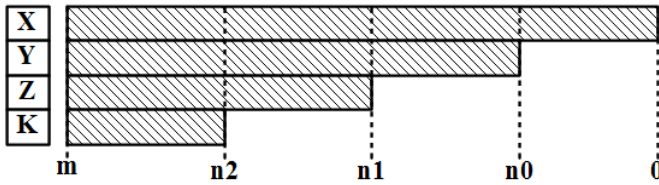


Fig. 8: Marks between numbers in the example of summing four components in signed area.

From the Eq. 2 of summing two components in signed area, it can be extended for the general case of many components. If R is the summing result of X, Y, Z, and K, then the formula of R can be described as:

$$\begin{aligned}
 R_{n0-1} &= R_{n0-2} = \dots = R_0 = X_{n0} \\
 R_{n0} &= X_{n0}(xor)Y_{n0} \\
 R_{n0+1} &= R_{n0+2} = \dots = R_{n1-1} = X_{n0}(or)Y_{n0} \\
 R_{n1} &= R_{n1-1}(xor)Z_{n1} \\
 R_{n1+1} &= R_{n1+2} = \dots = R_{n2-1} = R_{n1-1}(or)Y_{n1} \\
 R_{n2} &= R_{n2-1}(xor)K_{n2} \\
 R_{n2+1} &= R_{n2+2} = \dots = R_m = R_{n2-1}(or)K_{n2}
 \end{aligned} \quad (3)$$

where, $n0$, $n1$, and $n2$ are the marks of each of Y, Z, K to X. And m is the maximum bits of X, Y, Z, and K. Fig. 8 described the position of $n0$, $n1$, $n2$, and m in the example of summing four components in signed area. For example, with $X = 3FFF_h$, $Y = FFF_h$, $Z = 7F_h$, $K = 7_h$, and $n0 = 6$, $n1 = 11$, $n2 = 15$, $m = 18$, the result or R equals to $377BF_h$.

2) *LSB and MSB Areas Computation*: As mentioned above, MWTA uses 2-2 and 3-2 compressors with half CLA as its basic elements for the implementation. MWTA algorithm based on six principles:

- 1) Only 3 rows are computed once a time. After a 3-row process, other rows below and the result from the previous process make the next 3-row process. If the M is the number of partial products, there are $M - 2$ process times.
- 2) The signed bits are computed separately by signed area computation. The result of the signed area is added later in the end of the LSB and MSB computation process.
- 3) A 3-2 compressor is deployed to compute a column of 3 bits as shown in Fig. 9(a).
- 4) A 2-2 compressor computes a 2-bit column on the left-end of the partial products, the MSB-bit, as shown in Fig. 9(b).
- 5) Half CLA computes a column on the right-end of the partial products, the LSB-bit, as shown in Fig. 9(c).
- 6) The column of 1-bit (usually the multiplier result) is skipped in the MWTA process.

Fig. 10 shows an example of 8-bit multiplier. In Fig. 10, Z0, Z1, Z2, Z3, and Z4 are the partial products, and CZ0, CZ1, CZ2, and CZ3 are the complement carry from MBE in the previous step. The CZ4 is a special case when it is generated by the left-end block of 3-bit, a block that consists only one MSB-bit of the multiplicand. Therefore, CZ4 has two cases of 000_b or 001_b depends on the left-end bit of the multiplicand. The M in Fig. 10 is the result of signed area computation. Based on the example of 8-bit multiplier, the delay of the MWTA can be proven that it approximates to one 16-bit CLA.

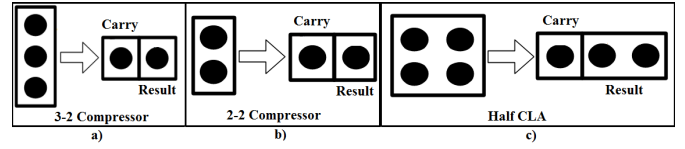


Fig. 9: Notations of 3-2 compressor (a), 2-2 compressor (b), and Half CLA (c).

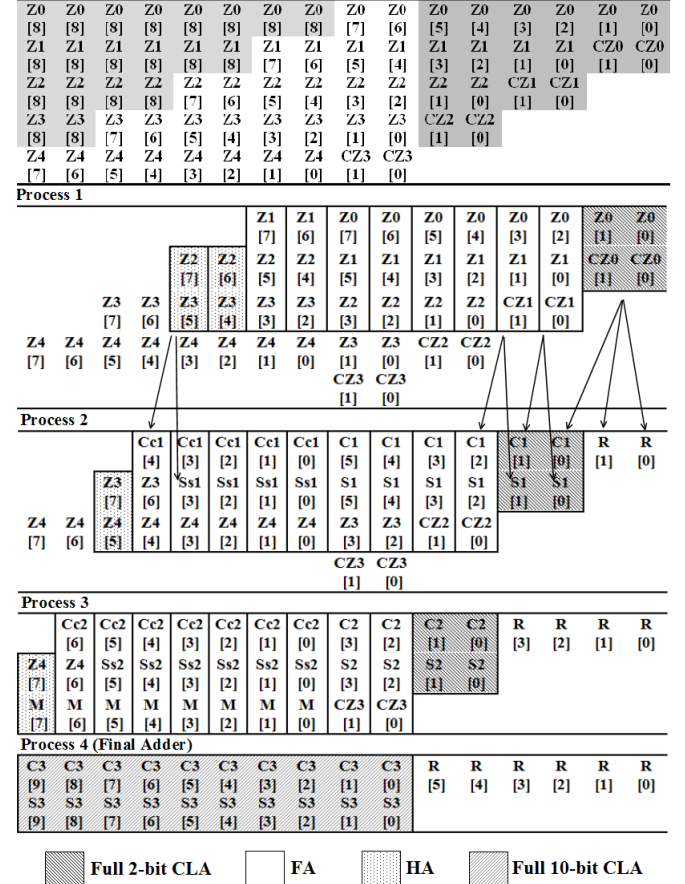


Fig. 10: Example MWTA process for 8-bit multiplier.

From Eq. 3, the time for M -computation is 3 times XOR gate delay plus 2 times OR gate delay. As shown in Fig. 10, in each process the design of LSB and MSB areas computation which consists of 3-2 compressor, 2-2 compressor, and half CLA is a parallel design. Then, the critical path from process 1 to process 2 equals to the delay of a half CLA. The same delay can be seen on the critical path from process 2 to process 3, and from process 3 to process 4. It is clear that the delay of M -computation is smaller than the delay from process 1 to process 3, which equals to 2 times delay of a half CLA. Called T_{HCLA} and T_{FCLA} are the delay of 2-bit half CLA and 4-bit full CLA. Based on the design in Fig. 6, the critical path of the proposed MWTA is $4T_{HCLA} + 2T_{FCLA} \approx 4T_{FCLA}$, which approximates to the delay of a 16-bit CLA. As mentioned in the subsection above, the delay of the first step in multiplier design equals to NOT gates delay only. Altogether, the total delay of the entire multiplier design approximates to a 16-bit CLA for the example of 8-bit multiplier. Similarly, a 32-bit multiplier design which deploys our proposed method can achieve the processing time approximated to the time delay of a 64-bit CLA.

TABLE II: The proposed design applied for radix-16 and radix-4.

		Radix-16		Radix-4	
		Area	Timing	Area	Timing
Partial Products Generation		33x8 FA + 8x33 NOT + 33 OR	1 T_{NOT} + 33 T_{FA}	16x33 NOT + 33 OR	1 T_{NOT}
Partial Product Addition	Process 1	32 FA + 4 HA + 1 Half 4-bit CLA	1.5 T_{HCLA}	30 FA + 2 HA + 1 Half 2-bit CLA	1 T_{HCLA}
	Process 2	33 FA + 3 HA + 1 Half 4-bit CLA	1.5 T_{HCLA}	31 FA + 1 HA + 1 Half 2-bit CLA	1 T_{HCLA}
	Process 3	33 FA + 3 HA + 1 Half 4-bit CLA	1.5 T_{HCLA}	31 FA + 1 HA + 1 Half 2-bit CLA	1 T_{HCLA}
	Process 4	33 FA + 4 HA + 1 Half 4-bit CLA	1.5 T_{HCLA}	31 FA + 1 HA + 1 Half 2-bit CLA	1 T_{HCLA}
	Process 5	33 FA + 3 HA + 1 Half 4-bit CLA	1.5 T_{HCLA}	31 FA + 1 HA + 1 Half 2-bit CLA	1 T_{HCLA}
	Process 6	33 FA + 3 HA + 1 Half 4-bit CLA	1.5 T_{HCLA}	31 FA + 1 HA + 1 Half 2-bit CLA	1 T_{HCLA}
	Process 7	33 FA + 3 HA + 1 Half 4-bit CLA	1.5 T_{HCLA}	31 FA + 1 HA + 1 Half 2-bit CLA	1 T_{HCLA}
	Process 8	N/A	N/A	31 FA + 1 HA + 1 Half 2-bit CLA	1 T_{HCLA}
	Process 9	N/A	N/A	31 FA + 1 HA + 1 Half 2-bit CLA	1 T_{HCLA}
	Process 10	N/A	N/A	31 FA + 1 HA + 1 Half 2-bit CLA	1 T_{HCLA}
	Process 11	N/A	N/A	31 FA + 1 HA + 1 Half 2-bit CLA	1 T_{HCLA}
	Process 12	N/A	N/A	31 FA + 1 HA + 1 Half 2-bit CLA	1 T_{HCLA}
	Process 13	N/A	N/A	31 FA + 1 HA + 1 Half 2-bit CLA	1 T_{HCLA}
	Process 14	N/A	N/A	31 FA + 1 HA + 1 Half 2-bit CLA	1 T_{HCLA}
	Process 15	N/A	N/A	31 FA + 1 HA + 1 Half 2-bit CLA	1 T_{HCLA}
Final Adder		9 Full 4-bit CLA	9 T_{FCLA}	1 Half 2-bit CLA + 8 Full 4-bit CLA	1 T_{HCLA} + 8 T_{FCLA}
Summarize		9 Full 4-bit CLA + 494 FA + 22 HA + 264 NOT + 32 OR + 7 Half 4-bit CLA	1 T_{NOT} + 33 T_{FA} + 10.5 T_{HCLA} + 9 T_{FCLA}	16 Half 2-bit CLA + 8 Full 4-bit CLA + 464 FA + 528 NOT + 32 OR	1 T_{NOT} + 16 T_{HCLA} + 8 T_{FCLA}

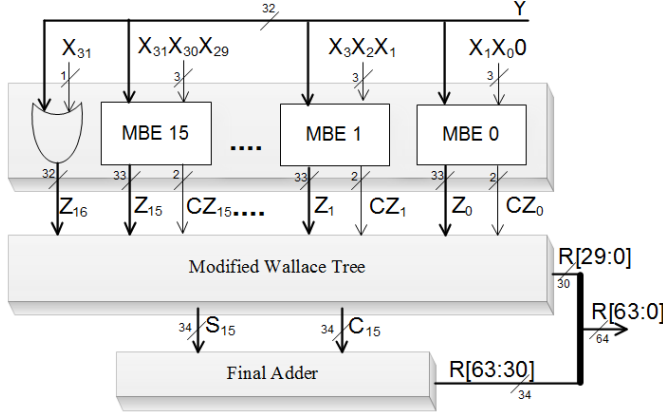


Fig. 11: The overview architecture of the proposed high-speed unsigned 32-bit multiplier design.

III. EXPERIMENTAL RESULTS

Fig. 11 shows the architecture of the proposed unsigned 32-bit multiplier. Table II gives a clear comparison between radix-4 and radix-16 designs. It can be seen that the number of partial products which are created by radix-16 design, 8 products with 7 summing process, is at half of those are created by radix-4 design, 16 products with 15 summing process. Although the area performances of both approaches are equivalent, the radix-4 design has much smaller processing time compared to the delay of the radix-16 design. To conclude, the proposed architecture has the best performances while it is implemented on radix-4 Booth encoder algorithm.

According to the clear comparison between radix-4 and radix-16 approaches, the radix-4 architecture is chosen for the implementation. The proposed multiplier design is implemented on DE2-115 Altera development kit with Cyclone IV FPGA chip in order to verify the functionality. Then, it is synthesized using a 90nm technology. The experimental results are compared with other designs as shown in Table III.

TABLE III: The experimental results comparison between the proposed design with other designs.

Multiplier design	Width of multiplier	ALUTs	Delay (ns)
Design in [9]	32	2704	9.54
Array multiplier [8]	32	1744	20.64
Wallace multiplier [8]	32	1905	11.60
Dadda multiplier [8]	32	1782	10.97
Reduced-area multiplier [8]	32	1802	12.03
Radix-4 Booth encoding [8]	32	1450	12.27
Proposed design	32	1788	4.13

The design in [9] uses 5-2 and 4-2 compressors to optimize the Wallace tree adder. Wallace multiplier [8] and Dadda multiplier in [8] are implemented by the Wallace and Dadda tree adder techniques, respectively. The reduced-area design in [8] combines Wallace and Dadda approaches to minimize the resources. Radix-4 Booth encoding multiplier in [8] is based on the radix-4 Booth algorithm. It can be seen that the proposed multiplier has a slightly increase in resources compared to those of the radix-4 design in [8]. But the delay of the proposed method is 3 times better. Thus, the proposed architecture can achieve the minimum delay according to Table III, while the resources are equivalent with those of the others.

The resources of FPGA-based implementation are 1788 ALUTs. And the area and timing on ASIC-based circuit are 58.28 mm^2 and 4.13 ns (equivalent to 242.13MHz maximum frequency), respectively.

IV. CONCLUSION AND FUTURE WORK

The paper has proposed a high-speed unsigned 32-bit multiplier. The proposed design advances from the modified radix-4 Booth encoder and modified Wallace tree adder. Owing to the modified radix-4 method moves the complement carry into the Wallace tree adder, the processing time is improved significantly compared to the traditional Booth encoder. The modified Wallace tree adder sums all the partial products with

their complement carry to find the final multiplier result. It is done by dividing the summing process into three computation areas: signed area, MSB area, and LSB area. The signed area is computed separately with MSB and LSB areas computation. In the MSB and LSB areas computation, a parallel architecture was designed to find the optimum delay. As a result, the proposed multiplier design can be embedded in various high-speed hardware implementations.

Due to the advantage of processing time, the proposed multiplier can be applied for many implementations which require high-speed property. In the future, a DSP core will be developed using the multiplier design in this paper. There are many IP cores in the DSP architectures that need a high-speed multiplier such as floating-point unit, arithmetic logic unit, FIR filter, audio/image processing, etc.

ACKNOWLEDGEMENT

This work was granted under Project 39/2013/HĐ-SKHCN by the Department of Science and Technology of Ho Chi Minh City.

REFERENCES

- [1] A. D. Booth, "A Signed Binary Multiplication Technique," *Quarterly J. Mechanical and Applied Math.*, Vol. 4, pp. 236-240, 1951.
- [2] C. S. Wallace, "A Suggestion for a Fast Multiplier," *IEEE Trans. Computers*, Vol. 13, No. 2, pp. 14-17, 1964.
- [3] L. Dadda, "Some Schemes for Parallel Multiplier," *Alta Frequenza*, Vol. 34, pp. 349-356, 1965.
- [4] A. Weinberger, "4:2 Carry-Save Adder Module," *IBM Technical Disclosure Bull.*, Vol. 23, 1981.
- [5] Z. Huang and M. Ercegovic, "High-Performance Left-to-Right Array Multiplier Design," in *Proc. 16th Symp. Computer Arithmetic*, pp. 4-11, June 2003.
- [6] J. P. Wang, S. R. Kuang, and S. C. Liang, "High-accuracy fixed-width modified Booth multipliers for lossy applications," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, Vol. 19, No. 1, pp. 52-60, Jan. 2011.
- [7] D. Villeger and V.G. Oklobdzija, "Analysis of Booth Encoding Efficiency in Parallel Multipliers Using Compressors for Reduction of Partial Products," *Proc. IEEE 27th Asilomar Conf. Signals, Systems, and Computer*, Vol. 1, pp. 781-784, 1993.
- [8] L. H. Hiung and K. L. S. Swee, "Performance Comparison Review of 32-Bit Multiplier," *Proc. IEEE 4th Int. Conf. on Intelligent and Advanced Systems (ICIA S2012)*, Vol. 2, pp. 836 - 841, 2012.
- [9] M. J. Rao and S. Dubey, "A High Speed and Area Efficient Booth Recoded Wallace Tree Multiplier for Fast Arithmetic Circuits," *Proc. IEEE Asia Pacific Conf. on Postgraduate Research in Microelectronics and Electronics (PrimeAsia)*, pp. 220 - 223, 2012.
- [10] F. Elguibaly, "A Fast Parallel Multiplier-Accumulator Using the Modified Booth Algorithm," *IEEE Trans. on Circuits and Systems II: Analog and Digital Signal Processing*, Vol. 47, pp. 902-908, 2000.
- [11] M. R. Meher, C. C. Jong, and Chip-Hong Chang, "A High Bit Rate Serial-Serial Multiplier With On-the-Fly Accumulation by Asynchronous Counters," *IEEE Trans. on Very Large Scale Integration (VLSI) Systems*, Vol. 19, pp. 1733-1745, 2011.
- [12] J. Y. Kang and J. L. Gaudiot, "A Fast and Well Structured Multiplier," *EUROMICRO Symp. Digital System Design*, 508-515, Aug. 2004.